

Pandas Transformations Handbook — Descriptive Edition

Each section explains *what* the transformation does, *when* to use it, and shows a small, concrete example with an output preview.

1. Importing and Creating DataFrames

What it does: Create a DataFrame from common sources like dicts, lists, CSV/JSON. This is the starting point for almost all transformations.

When to use: Use when you need to load data into memory for analysis or transformation.

Example:

```
import pandas as pd data = {'Name': ['Alice', 'Bob'], 'Age': [25, 32]} df = pd.DataFrame(data) # From CSV: df = pd.read_csv('people.csv')
```

Output preview:

```
Name Age Alice 25 Bob 32
```

2. Selecting Data (columns/rows/conditions)

What it does: Subsets your data by columns, by row labels/positions, or by conditions. Returns Series/DataFrame as needed.

When to use: Use to focus on specific fields or records before further processing.

Example:

```
# Columns df['Name'] # Series df[['Name', 'Age']] # DataFrame # Rows by label/position df.loc[0] # first row by label df.iloc[0] # first row by position # Conditional filter adults = df[df['Age'] >= 30]
```

Output preview:

```
adults: Name Age Bob 32
```

3. Renaming Columns and Index

What it does: Gives meaningful names to columns and/or index. Improves readability and downstream joins.

When to use: Use after ingest when column names are cryptic or inconsistent.

Example:

```
df = df.rename(columns={'Name': 'full_name', 'Age': 'age'}) df = df.rename(index={0: 'row0'})
```

Output preview:

```
Columns: full_name, age Index: row0, 1
```

4. Adding / Updating Columns

What it does: Creates new derived columns or updates existing ones using vectorized operations or functions.

When to use: Use to engineer features for modeling or to compute business KPIs.

Example:

```
df['age_plus_10'] = df['age'] + 10 df['segment'] = df['age'].apply(lambda x: '35+'  
if x >= 35 else '<35')
```

Output preview:

```
full_name age age_plus_10 segment Alice 25 35 <35 Bob 32 42 <35
```

5. Removing Columns and Rows

What it does: Drops irrelevant columns/rows to simplify the dataset.

When to use: Use to avoid noise, reduce memory, or remove temporary fields.

Example:

```
df = df.drop(columns=['age_plus_10']) df = df.drop(index='row0')
```

Output preview:

```
Remaining rows/cols after drop: full_name age Bob 32
```

6. Handling Missing Data (dropna/fillna)

What it does: Removes or imputes missing values to keep downstream operations stable.

When to use: Use dropna for strict analyses; use fillna to impute values (mean/median/mode or constants).

Example:

```
df['age'] = df['age'].fillna(df['age'].median()) clean =  
df.dropna(subset=['full_name'])
```

Output preview:

```
Missing ages replaced by median; rows with empty names removed.
```

7. Sorting Data (sort_values/index)

What it does: Orders rows by one or more columns or by index, enabling ranked views and top-N queries.

When to use: Use to find top performers, latest dates, or to prepare for diff/shift operations.

Example:

```
df.sort_values(by=['age'], ascending=False, inplace=True)  
df.sort_index(inplace=True)
```

Output preview:

```
Sorted by age descending: full_name age Bob 32 Alice 25
```

8. Changing Data Types (astype/to_datetime)

What it does: Converts columns to proper dtypes (numeric, categorical, datetime) for correctness and performance.

When to use: Use before math/time ops or to reduce memory with categories/ints.

Example:

```
df['age'] = df['age'].astype('int64') df['joined'] =  
pd.to_datetime(['2024-06-01', '2024-12-15'])
```

Output preview:

```
age dtype: int64 joined dtype: datetime64[ns]
```

9. Grouping & Aggregation (groupby/agg)

What it does: Aggregates rows into groups to compute statistics like sum/mean/max per group.

When to use: Use for rollups: sales by region, users by cohort, etc.

Example:

```
sales = pd.DataFrame({ 'Region':['E','E','W','W'], 'Amount':[100, 80, 120, 60] })
out = sales.groupby('Region').agg(total=('Amount','sum'), avg=('Amount','mean'))
```

Output preview:

```
out: Region total avg E 180 90.0 W 180 90.0
```

10. Pivot Tables

What it does: Summarizes data into a matrix (index x columns) with an aggregation function.

When to use: Use for cross-tab reports like revenue by (region x product).

Example:

```
pd.pivot_table(sales, values='Amount', index='Region', columns=None, aggfunc='sum',
fill_value=0)
```

Output preview:

```
Region Amount E 180 W 180
```

11. Merging & Joining (merge)

What it does: Combines two DataFrames horizontally using keys (SQL-style joins).

When to use: Use to enrich a fact table with dimensions or lookup tables.

Example:

```
users = pd.DataFrame({'id':[1,2], 'name':['Alice','Bob']}) salary =  
pd.DataFrame({'id':[1,2], 'salary':[70_000, 90_000]}) merged = users.merge(salary,  
on='id', how='left')
```

Output preview:

```
merged: id name salary 1 Alice 70000 2 Bob 90000
```

12. Concatenating (concat)

What it does: Stacks DataFrames vertically (more rows) or horizontally (more columns).

When to use: Use to append monthly files (axis=0) or combine unrelated feature sets (axis=1).

Example:

```
north = pd.DataFrame({'city':['NY'], 'sales':[120]}) south =  
pd.DataFrame({'city':['LA'], 'sales':[100]}) rows = pd.concat([north, south],  
axis=0, ignore_index=True)
```

Output preview:

```
rows: city sales NY 120 LA 100
```

13. Reshaping (melt/pivot)

What it does: melt: wide→long; pivot: long→wide. Restructure columns and rows.

When to use: Use to normalize denormalized tables or to prepare input for plotting/ML.

Example:

```
wide = pd.DataFrame({'name':['A','B'], 'q1':[10,15], 'q2':[12,14]}) long =  
wide.melt(id_vars='name', var_name='quarter', value_name='value') back =  
long.pivot(index='name', columns='quarter', values='value').reset_index()
```

Output preview:

```
long: name quarter value A q1 10 A q2 12 B q1 15 B q2 14
```

14. String Operations (.str)

What it does: Vectorized text cleaning and pattern searches on object/string dtype.

When to use: Use for case normalization, extracting patterns, or replacing text.

Example:

```
cust = pd.Series([' alice ', 'BOB', 'charlie']) clean = cust.str.strip().str.title()  
mask = clean.str.contains('Bo', case=False)
```

Output preview:

```
clean: ['Alice', 'Bob', 'Charlie'] mask: [False, True, False]
```

15. Applying Functions (apply/applymap)

What it does: apply: row/column-wise; applymap: element-wise for entire DataFrame.

When to use: Use when a vectorized method is unavailable; keep functions fast to avoid slow loops.

Example:

```
df['age_sq'] = df['age'].apply(lambda x: x*x) upper =  
df[['full_name']].applymap(lambda x: x.upper())
```

Output preview:

```
age_sq example: 25→625; 32→1024 upper: FULL_NAME column uppercased
```

16. Window Functions (rolling/expanding/cumsum/shift)

What it does: Compute metrics over a moving window or cumulatively; shift aligns previous rows.

When to use: Use for moving averages, running totals, lag features.

Example:

```
ts = pd.Series([10,20,30,40]) roll = ts.rolling(2).mean() # [NaN,15,25,35] run =  
ts.cumsum() # [10,30,60,100] lag1 = ts.shift(1) # [NaN,10,20,30]
```

Output preview:

```
roll: NaN, 15, 25, 35 run: 10, 30, 60, 100 lag1: NaN, 10, 20, 30
```

17. Sampling (sample)

What it does: Draws a random subset of rows by count or fraction.

When to use: Use for quick EDA or to create train/test splits (with random_state).

Example:

```
sampled = df.sample(n=1, random_state=42) half = df.sample(frac=0.5,  
random_state=42)
```

Output preview:

```
Random 1 row; 50% sample with reproducibility.
```

18. Exporting (to_csv/to_excel)

What it does: Writes DataFrame to files for sharing or downstream tools.

When to use: Use to persist outputs of pipelines or analysis snapshots.

Example:

```
df.to_csv('output.csv', index=False) df.to_excel('output.xlsx', index=False)
```

Output preview:

```
Creates files without the index column.
```

19. JSON Handling (to_json/read_json)

What it does: Serializes to/from JSON for APIs and nested data.

When to use: Use orient='records' for row-wise list of dicts, great for web services.

Example:

```
df.to_json('data.json', orient='records') df2 = pd.read_json('data.json')
```

Output preview:

```
data.json contains: [{'full_name': 'Alice', 'age': 25}, ...]
```

20. Date & Time Operations (.dt)

What it does: Accessors for extracting parts of datetimes and performing time arithmetic.

When to use: Use to roll up by year/month, compute weekday, or filter by ranges.

Example:

```
df['year'] = df['joined'].dt.year df['month'] = df['joined'].dt.month df['weekday']  
= df['joined'].dt.day_name()
```

Output preview:

```
year: 2024/2024; month: 6/12; weekday: Saturday/Sunday (examples)
```

21. Conditional Columns (np.where)

What it does: Creates fast conditional values (vectorized if-else).

When to use: Use for flags/labels without writing Python loops.

Example:

```
import numpy as np df['is_senior'] = np.where(df['age'] >= 35, 1, 0)
```

Output preview:

```
is_senior: 0 for age<35; 1 for age>=35
```

22. Duplicates (duplicated/drop_duplicates)

What it does: Finds or removes duplicate rows based on subset of columns.

When to use: Use to deduplicate IDs before joins or reporting.

Example:

```
dupe_mask = df.duplicated(subset=['full_name'], keep='first') dedup =  
df.drop_duplicates(subset=['full_name'])
```

Output preview:

```
dupe_mask example: [False, True, ...] dedup keeps first occurrence only.
```

23. MultiIndex Operations (set_index/reset_index)

What it does: Creates hierarchical indexes enabling compact storage and grouped selection.

When to use: Use when grouping by multiple keys or reshaping between levels.

Example:

```
m = df.set_index(['segment', 'full_name']) row = m.loc[('~35', 'Alice')] if  
('~35', 'Alice') in m.index else None flat = m.reset_index()
```

Output preview:

```
MultiIndex rows accessible by tuple keys; reset_index returns regular columns.
```

24. Advanced Filtering (query)

What it does: SQL-like string queries with boolean expressions and @variables.

When to use: Use for readable filters, especially with many conditions.

Example:

```
threshold = 30 filtered = df.query('age >= @threshold and segment == "<35"')
```

Output preview:

```
Rows with age>=30 and segment <35.
```

25. Binning (cut/qcut)

What it does: cut: fixed bins; qcut: quantile-based bins (equal-sized groups).

When to use: Use to create age bands, score buckets, or deciles for modeling.

Example:

```
df['age_band'] = pd.cut(df['age'], bins=[0,18,35,60],
labels=['Teen','Adult','Senior']) df['decile'] = pd.qcut(df['age'], q=10,
labels=False)
```

Output preview:

```
age_band: Teen/Adult/Senior decile: integers 0..9
```

26. Categorical Data (astype('category'))

What it does: Converts strings to categorical dtype for memory savings and faster ops on repeated labels.

When to use: Use for low-cardinality dimensions (e.g., country, segment).

Example:

```
df['segment'] = df['segment'].astype('category')
```

Output preview:

```
segment dtype: category (saves memory)
```

27. Value Counts & Crosstab

What it does: Counts unique values and builds contingency tables.

When to use: Use for quick distributions and frequency analysis.

Example:

```
df['segment'].value_counts(dropna=False) pd.crosstab(df['segment'], df['year'])
```

Output preview:

```
value_counts: {'<35': 2} crosstab: rows=segment, cols=year counts
```

28. Replace & Map (replace/map)

What it does: replace: literal substitutions; map: dict/Series mapping (can introduce NaN for missing keys).

When to use: Use to standardize labels or map codes to descriptions.

Example:

```
df['segment'] = df['segment'].replace({'<35':'Under35'}) tier_map =
{'Under35':'Silver', '35+':'Gold'} df['tier'] = df['segment'].map(tier_map)
```

Output preview:

```
segment: 'Under35'; tier: 'Silver'
```

29. Explode (explode)

What it does: Transforms a list-like column into multiple rows, one element per row.

When to use: Use after splitting tags/skills arrays into tidy rows.

Example:

```
skills = pd.DataFrame({'name':['Alice','Bob'], 'skills':[ ['SQL','Python'],
['Excel'] ]}) tidy = skills.explode('skills', ignore_index=True)
```

Output preview:

```
tidy: name skills Alice SQL Alice Python Bob Excel
```

30. Apply Row-Wise with axis=1

What it does: Compute a value from multiple columns for each row.

When to use: Use for rule-based scoring or conditional concatenations.

Example:

```
def score(row): return (row['age'] >= 30) + (row['segment'] == '<35') df['score'] = df.apply(score, axis=1)
```

Output preview:

```
score = 0/1/2 depending on conditions
```

Tip: Prefer vectorized operations over row-wise apply for performance. Convert text dates to datetime early, and choose appropriate dtypes to save memory.